National College of Ireland

Cloud DevOpsSec

An Introduction

Cloud DevOpsSec

- ***DevOps*** – brings together development and operations teams during the software development lifecycle

- ***DevOpsSec*** – currently, the trend is to merge security with DevOps throughout the process

- ***Cloud DevOpsSec*** – cloud-based technologies and services are widely used
  - According to Gartner, in 2025, the worldwide end-user spending on public cloud services was forecasted to total $723.4 billion. In 2024, end-user spending on public cloud services was $595.7 billion.

DevOps

- DevOps is *“an engineering culture of collaboration, ownership, and learning with the purpose of* ***accelerating the software development life cycle*** *from ideation to production”* (Freeman, 2019)

DevOps

- DevOps prioritizes people over process and process over tooling
- DevOps enables to deliver value and adapt to market needs faster and at scale

DevOpsSec

- DevOpsSec automates security within the DevOps
  - Code Analysis
  - Secure programming
  - Logging
  - Event monitoring
  - Vulnerability assessment
- Shared responsibility model/ ownership of security

Cloud DevOpsSec

Cloud Computing – An Introduction

- Essential Characteristics
  - On-demand self-service
  - Resource pooling
  - Rapid elasticity
  - Measured Service
  - Broad network access

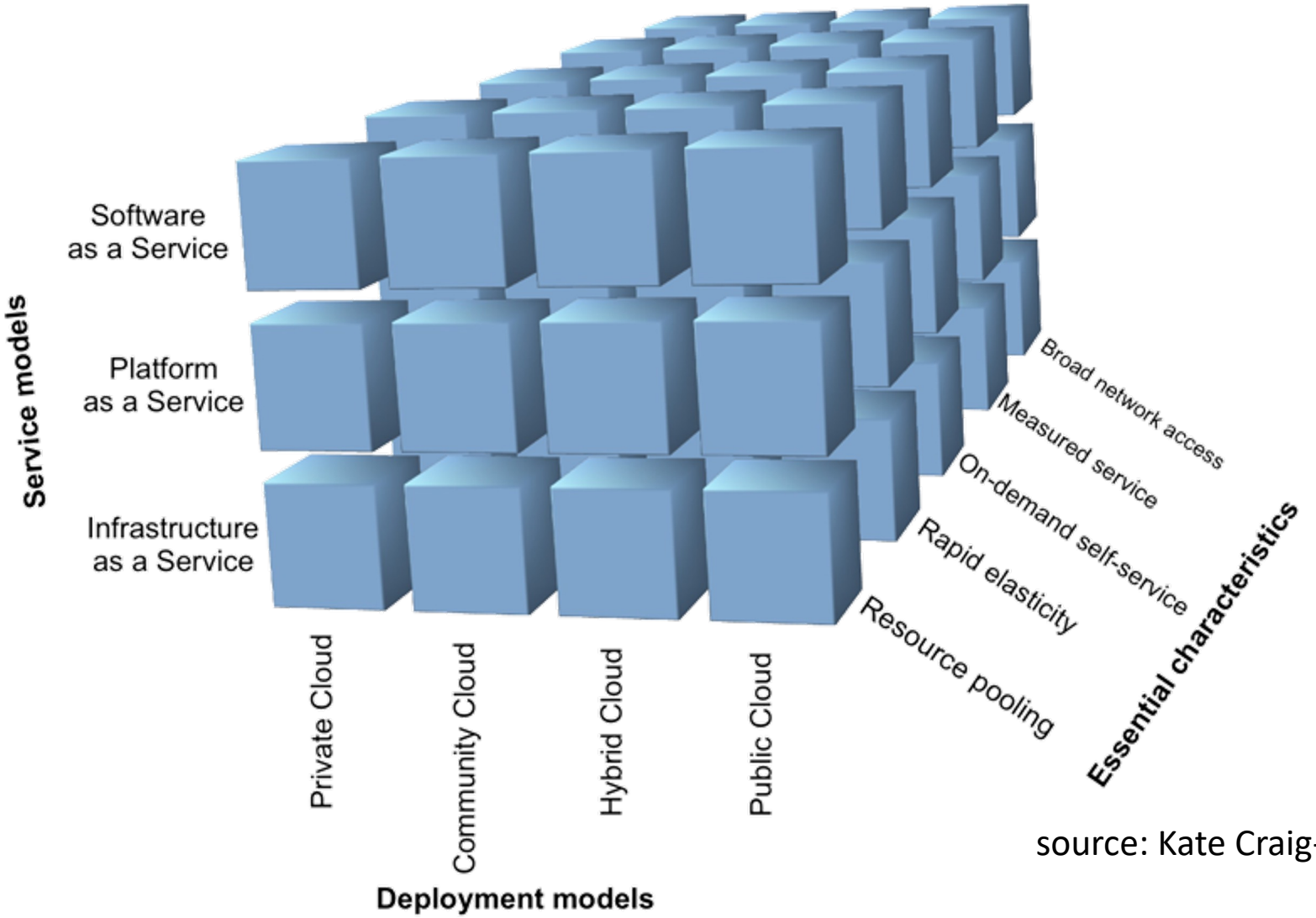
Cloud Computing – An Introduction

- Service Models
  - Infrastructure as a Service
  - Platform as a Service
  - Software as a Service

Cloud Computing – An Introduction

- Deployment Models
  - Private Cloud
  - Community Cloud
  - Hybrid Cloud
  - Public Cloud

Cloud Computing – An Introduction



source: Kate Craig-Wood, NIST, and G-Cloud

Cloud ***Dev***OpsSec

Python

- General purpose, high-level, object-oriented programming language
- A dynamically typed language
- An interpreted language
- Is a portable language
  - Runs on many types of Unix, including Linux and macOS, and Windows

Python

- Has exception handling features, like Java, C#, Ruby, to make it easy to handle errors
- Has automatic memory management
- Has an extensive standard library

Python

- Suitable for Rapid Application Development
- It supports multiple programming paradigms beyond object-oriented programming, such as procedural and functional programming
- Can also be used as a scripting language

Object-Oriented Programming

- A programming paradigm using
  - Classes and Objects
  - Data Members/Attributes
  - Methods

Python 101 - Class Definition Syntax

```
class ClassName:
    <statement-1>
    .
    .
    .
    <statement-N>
```

source: https://docs.python.org/3.8/tutorial/classes.html

- Note that the body of the class is indented
- In Python we use indentation to group statements in the same block of code

Python 101 – Student class – An Example

```
class Student:

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id
```

Python 101 – Student class – An Example

- __init__ method is similar with the constructor in other languages (e.g. Java, C#)
  - It initializes an instance with the provided arguments
  - The first argument is the instance – it is passed in *automatically* when the method is called
    - Ensure you don't forget to specify that parameter!

Python 101 – Student class – An Example

- __init__ method is similar with the constructor in other languages (e.g. Java, C#)
  - It initializes an instance with the provided arguments
  - The first argument is the instance – it is passed in *automatically* when the method is called
    - Ensure you don't forget to specify that parameter!

Creating (Instantiating) a Student

```
s1 = Student('Emma', 'Smith', 'x123456789')
s2 = Student('Arjun', 'Kumar', 'x987654321')

print(s1.first_name)
print(s1.student_id)
```

Creating (Instantiating) a Student

```
s1 = Student('Emma', 'Smith', 'x123456789')
s2 = Student('Arjun', 'Kumar', 'x987654321')

print(s1.first_name)
print(s1.student_id)
```

- Output
  - Emma
  - x123456789

Python 101 – Student class – An Example (cont.)

```
class Student:

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id

    def greetings(self):
        return 'My name is {}, I\'m a student.\n'.format(self.first_name)
```

Python 101 – Student class – An Example (cont.)

```
class Student:

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id

    def greetings(self):
        return 'My name is {}, I\'m a student.\n'.format(self.first_name)
```

- Note that each instance method we declare within a class, must have the first parameter reserved for the instance

Python 101 – Using the Student class – An Example (cont.)

```
class Student:

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id

    def greetings(self):
        return 'My name is {}, I\'m a student.\n'.format(self.first_name)


s1 = Student('Emma', 'Smith', 'x123456789')
s2 = Student('Arjun', 'Kumar', 'x987654321')

print (s1.greetings())
```

- Note that each instance method we declare within a class, must have the first parameter reserved for the instance
- When the method is called, the instance is automatically passed as the first argument
  - i.e. we don’t have to provide an argument for the first parameter
    - e.g. s1.greetings()

Class and Instance Variables

- Instance variables
  - When an instance/object is created it has its own distinct copy of the instance variables
- Class variables contain data shared by all instances/objects of a classs

Class and Instance Variables – An Example

```
class Student:

    count = 0

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id
        Student.count += 1
```

Class and Instance Variables – An Example

```
class Student:

    count = 0

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id
        Student.count += 1
```

```
s1 = Student('Emma', 'Smith', 'x123456789')
s2 = Student('Arjun', 'Kumar', 'x987654321')
s3 = Student('Omar', 'Omar', 'x111111111')

print(Student.count)
```

Class and Instance Variables – An Example

```
class Student:

    count = 0

    def __init__(self, first_name, last_name, student_id):
        self.first_name = first_name
        self.last_name = last_name
        self.student_id = student_id
        Student.count += 1
```

```
s1 = Student('Emma', 'Smith', 'x123456789')
s2 = Student('Arjun', 'Kumar', 'x987654321')
s3 = Student('Omar', 'Omar', 'x111111111')

print(Student.count)
```

- What would be the count value displayed?

Class and Instance Variables

- Class variables contain data shared by all instances/objects of a classs
- Access a class variable either
  - Via the class itself, OR
    - e.g. Student.count
  - Via an instance of the class
    - e.g. self.count

Methods in Python (in an OOP context)

- Instance methods
  - When the method is called, the instance is automatically passed as the first argument
- Class methods
  - Defined using @classmethod decorator
  - When the method is called, the class is automatically passed as the first argument

Methods in Python (in an OOP context)(cont.)

- Static methods
  - Defined using @staticmethod decorator
  - Only use static methods if you have to implement some functionality related to the class, but you don’t need to access the instance (i.e. *self*) or the class (i.e. *cls*) within that method!

Getter and Setter Methods in Python

- Getter method
  - Defined using @property

```
@property
def name(self):
    return '{} {}'.format(self.first_name, self.last_name)
```

- Setter method

```
@name.setter
def name(self, name):
    self.first_name, self.last_name = name.split(' ')
```

Inheritance – An Example

```
class Person:

    def __init__(self, first_name, last_name):
        self.first_name = first_name
        self.last_name = last_name

    def greetings(self):
         return 'My name is {}.\n'.format(self.first_name)



class Student(Person):

    def __init__(self, first_name, last_name, student_id):
        super().__init__(first_name, last_name)
        self.student_id = student_id

    def greetings(self):
        return 'My name is {}, I\'m a student.\n'.format(self.first_name)
```

Inheritance – General Syntax

```
class <DerivedClassName>(<BaseClassName>):
        <statement-1>
        …
        <statement-N>
```

- e.g. class Student(Person):
  - Student is the derived class
  - Person is the base or super class

Inheritance – General Syntax

- Python supports multiple inheritance

```
class <DerivedClassName>(<BaseClass1>, <BaseClass2>, …, <BaseClassN>):
        <statement-1>
        …
        <statement-N>
```

- e.g. class Student(Person, Employee):

Resources

- Gartner, 2025 https://www.gartner.com/en/newsroom/press-releases/2024-11-19-gartner-forecasts-worldwide-public-cloud-end-user-spending-to-total-723-billion-dollars-in-2025
- Emily Freeman (2019), DevOps For Dummies, 1st. 23, John Wiley & Sons, p.320

- Python
  - Download Python https://www.python.org/downloads/
  - Python Documentation https://docs.python.org/3/
  - The Python Tutorial https://docs.python.org/3/tutorial/index.html